

WEEK 3 – FINAL DESIGN DOCUMENT

(MLFQ + Boosting + Testing)

Project: Multi-Level Feedback Queue (MLFQ) Scheduler

Team Members: Marium Ali Lakhani (29183), Fatima Faisal (28989)

Date: 3rd Dec 2025

1. Introduction

This document summarizes the implementation and testing of Week 3. It includes Priority Boosting + Polishing for the xv6 MLFQ scheduler.

The goal was to:

- Prevent starvation using periodic priority boosting
- Polish queue logic + demotion rules
- Add optional test syscalls
- Compare fairness between CPU-bound and I/O-bound processes

2. MLFQ Overview

MLFQ has 4 queues:

- Q0: Highest priority (4-tick quantum)
- Q1: 8 ticks
- Q2: 16 ticks
- Q3: Lowest priority (runs until demoted/boosted)

Rules:

- New processes start in Q0

- Processes demote when they use full quantum
- I/O-bound processes return to Q0 naturally because they yield
- Every BOOST_TIME ticks, all processes are moved to Q0

3. Final Scheduler Implementation (Week 3)

```
int ticks_since_boost = 0;

struct proc_queue queues[QUEUE_COUNT];

//add to queue

void enqueue(struct proc_queue *q, struct proc *p){
    if ((q->tail + 1) % NPROC == q->head){ //circular queue sod and checking that its doesnt point
again to head

        panic("queue overflow");
    }

    q->procs[q->tail] = p;

    q->tail = (q->tail + 1) % NPROC;

    p->in_queue = 1;
}

struct proc *dequeue(struct proc_queue *q){
    if (is_empty(q)){ //empty queue

        return 0;
    }

    struct proc *p = q->procs[q->head]; //assigning p the process at the head pointer of the queue

    q->head = (q->head + 1) % NPROC;

    p->in_queue = 0;
}
```

```
    return p;
```

```
}
```

```
int is_empty(struct proc_queue *q){
```

```
    return q->head == q->tail;
```

```
}
```

```
void print_queue(void) {
```

```
    for (int i = 0; i < QUEUE_COUNT; i++) {
```

```
        struct proc_queue *q = &queues[i];
```

```
        acquire(&q->lock);
```

```
        printf("Queue %d: ", i);
```

```
        int idx = q->head;
```

```
        while (idx != q->tail) {
```

```
            struct proc *p = q->procs[idx];
```

```
            if (p != 0)
```

```
                printf("%d ", p->pid);
```

```
            idx = (idx + 1) % NPROC;
```

```
        }
```

```
void
```

```
scheduler(void)
```

```
{
```

```
    struct proc *p;
```

```
    struct cpu *c = mycpu();
```

```
c->proc = 0;
```

```
for(;;){
```

```
    intr_on();
```

```
    //iterate over queues from highest to lowest priorities
```

```
    for(int q = 0; q < QUEUE_COUNT; q++){
```

```
        struct proc *p;
```

```
        acquire(&queues[q].lock);
```

```
        p = dequeue(&queues[q]);
```

```
        release(&queues[q].lock);
```

```
        if(p){ //not null
```

```
            acquire(&p->lock);
```

```
            if(p->state == RUNNABLE){
```

```
                //run the process
```

```
                p->state = RUNNING;
```

```
                c->proc = p;
```

```
                swtch(&c->context, &p->context);
```

```
                c->proc = 0;
```

```
                //demotion or re-enqueue
```

```
                if(p->state == RUNNABLE){
```

```
                    int demoted = 0;
```

```

// Demotion rule:

// q==0 → 4 ticks

// q==1 → 8 ticks

// q==2 → 16 ticks

// q==3 → infinite

int timeslice = (q == 0 ? QUEUE1_TIME :
                 (q == 1 ? QUEUE2_TIME :
                  QUEUE3_TIME));

if(p->time_in_queue >= timeslice && p->queue < QUEUE_COUNT - 1){
    p->queue++;          // demote
    p->time_in_queue = 0;
    demoted = 1;

    printf("\nDemoted PID %d to Queue %d\n", p->pid, p->queue);
}

acquire(&queues[p->queue].lock);
enqueue(&queues[p->queue], p);
release(&queues[p->queue].lock);

if(demoted){
    print_queue();
}
}

}

release(&p->lock);
}

```

```
}
```

```
//boosting logic
```

```
if(ticks_since_boost >= BOOST_TIME){
```

```
    //boost
```

```
    for(int q = 1; q < QUEUE_COUNT; q++){
```

```
        acquire(&queues[q].lock);
```

```
        while(!is_empty(&queues[q])){
```

```
            struct proc *p = dequeue(&queues[q]);
```

```
            release(&queues[q].lock);
```

```
            if(!p) break;
```

```
            printf("\nboosting %d\n", p->pid);
```

```
            acquire(&p->lock);
```

```
            p->queue = 0;          //move to highest priority queue
```

```
            p->time_in_queue = 0;  //reset time
```

```
            release(&p->lock);
```

```
            acquire(&queues[0].lock);
```

```
            enqueue(&queues[0], p);
```

```
            release(&queues[0].lock);
```

```
            print_queue();
```

```

        acquire(&queues[q].lock);

    }

    release(&queues[q].lock);

}

ticks_since_boost = 0; //reset timer

}

//If there was nothing runnable in any queue, idle the CPU

int found = 0;

for(p = proc; p < &proc[NPROC]; p++){

    acquire(&p->lock);

    if(p->state == RUNNABLE){

        found = 1;

        release(&p->lock);

        break;

    }

    release(&p->lock);

}

if(found == 0){

    asm volatile("wfi");

}

}

}

```

IN TRAP.C usertrap(): if(which_dev == 2){

p->time_in_queue++; //when timer interrupt occurs increment time in queue

printf("\nprocess %d ticks: %d\n", p->pid, p->time_in_queue);

```

ticks_since_boost++; //and also time since boost

printf("ticks since boost: %d\n\n", ticks_since_boost);

//kiya process ne apna time slice poori kiya

int slice = (p->queue == 0 ? QUEUE1_TIME :

            p->queue == 1 ? QUEUE2_TIME : QUEUE3_TIME);

if((p->time_in_queue >= slice) || (ticks_since_boost >= BOOST_TIME)){

    yield();

}

}

prepare_return();

```

4. Boosting Logic Explanation

When `ticks_since_boost >= BOOST_TIME`:

1. For queues Q1, Q2, Q3
2. Dequeue every process
3. Reset:
 - `p->queue = 0`
 - `p->time_in_queue = 0`
4. Re-enqueue into Q0
5. Print queues
6. Reset `ticks_since_boost = 0`

This ensures:

- No starvation
- CPU-bound processes eventually rise back to Q0
- I/O-bound processes always remain responsive

5. Additional Features

Implemented `boostproc()` syscall

- Manually boosts only the calling process
- Used for debugging starvation

`boostall()` syscall

Calls full system-wide boosting

- Used to test correctness of your boost logic

```
void boost_all(void) {
    // 1) boost queued processes

    for (int q = 1; q < QUEUE_COUNT; q++) {
        acquire(&queues[q].lock);

        struct proc *bp;

        while ((bp = dequeue(&queues[q])) != 0) {
            acquire(&bp->lock);

            bp->queue = 0;

            bp->time_in_queue = 0;

            release(&bp->lock);

            acquire(&queues[0].lock);

            enqueue(&queues[0], bp);
        }
    }
}
```

```

        release(&queues[0].lock);
    }

    release(&queues[q].lock);
}

extern struct cpu cpus[]; extern int ncpu;

for (int i = 0; i < ncpu; i++) {

    struct cpu *cc = &cpus[i];

    struct proc *rp = cc->proc;

    if (rp == 0) continue;

    acquire(&rp->lock);

    if (rp->queue > 0) {

        rp->queue = 0;

        rp->time_in_queue = 0;

        if (!rp->in_queue) {

            acquire(&queues[0].lock);

            enqueue(&queues[0], rp);

            release(&queues[0].lock);

        }

    }

    release(&rp->lock);
}

ticks_since_boost = 0;
}

```

6. Testing Strategy

Test 1 – Basic Queue Functionality

Run on cmd: \$ burn

mariium@DESKTOP-0K07MJA: X + v

```
$ burn

fork enqueued 4
Queue 0:
Queue 1:
Queue 2:
Queue 3:
CPU-bound process 4 starting

process 4 ticks: 1
ticks since boost: 1

process 4 ticks: 2
ticks since boost: 2

process 4 ticks: 3
ticks since boost: 3

process 4 ticks: 4
ticks since boost: 4

Demoted PID 4 to Queue 1
Queue 0:
Queue 1: 4
Queue 2:
Queue 3:

process 4 ticks: 1
ticks since boost: 5

process 4 ticks: 2
ticks since boost: 6

process 4 ticks: 3
ticks since boost: 7

process 4 ticks: 4
ticks since boost: 8

process 4 ticks: 5
ticks since boost: 9

process 4 ticks: 6
ticks since boost: 10
```

Test 2 – Priority Boosting Works

Let burn run past your **BOOST_TIME** (40 ticks).

Process is being boosted to highest priority queue

```
mariu@DESKTOP-0K07MJA: × + v
ticks since boost: 37

process 4 ticks: 10
ticks since boost: 38

process 4 ticks: 11
ticks since boost: 39

process 4 ticks: 12
ticks since boost: 40

boosting 4
Queue 0: 4
Queue 1:
Queue 2:
Queue 3:

process 4 ticks: 1
ticks since boost: 1

process 4 ticks: 2
ticks since boost: 2

process 4 ticks: 3
ticks since boost: 3

process 4 ticks: 4
ticks since boost: 4

Demoted PID 4 to Queue 1
Queue 0:
Queue 1: 4
Queue 2:
Queue 3:

process 4 ticks: 1
ticks since boost: 5

process 4 ticks: 2
ticks since boost: 6

process 4 ticks: 3
ticks since boost: 7
```

Test 3 – CPU-bound vs I/O-bound fairness

Run: io

I/O-bound processes should always stay in Q0

CPU-bound processes from burn should drop to Q3 and later get boosted back.

Output:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh

fork enqueued 2
Queue 0: 2
Queue 1:
Queue 2:
Queue 3:
$ io

fork enqueued 3
Queue 0: 3
Queue 1:
Queue 2:
Queue 3:
I/O-bound process 3 starting
I/O process 3 iteration 0
I/O process 3 iteration 1
I/O process 3 iteration 2
I/O process 3 iteration 3
I/O process 3 iteration 4
I/O process 3 iteration 5
I/O process 3 iteration 6
I/O process 3 iteration 7
I/O process 3 iteration 8
I/O process 3 iteration 9
I/O-bound process 3 done
$ boostproc

fork enqueued 4
Queue 0: 4
Queue 1:
Queue 2:
Queue 3:
```

```
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh

fork enqueued 2
Queue 0: 2
Queue 1:
Queue 2:
Queue 3:
$ io

fork enqueued 3
Queue 0: 3
Queue 1:
Queue 2:
Queue 3:
I/O-bound process 3 starting
I/O process 3 iteration 0
I/O process 3 iteration 1
I/O process 3 iteration 2
I/O process 3 iteration 3
I/O process 3 iteration 4
I/O process 3 iteration 5
I/O process 3 iteration 6
I/O process 3 iteration 7
I/O process 3 iteration 8
I/O process 3 iteration 9
I/O-bound process 3 done
$

nv -f fs.img fs.img.bk
mvfs/mkfs fs.img README user/findtest.sh user/sidive.txt user/cat user/echo user
./forktest user/grep user/init user/kill user/ln user/ls user/mkdir user/rm
user/sh user/stressfs user/usertests user/grind user/wc user/zombie user/lo
stress user/_forphan user/_dorphan user/_pininfo user/_burn user/_io user/_quick us
r/_mdundump
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 195
total 2890
ballocc: first 1999 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3
nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format
raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh

fork enqueued 2
Queue 0: 2
Queue 1:
Queue 2:
Queue 3:
$ io

fork enqueued 3
Queue 0: 3
Queue 1:
Queue 2:
Queue 3:
I/O-bound process 3 starting
I/O process 3 iteration 0
I/O process 3 iteration 1
I/O process 3 iteration 2
I/O process 3 iteration 3
I/O process 3 iteration 4
I/O process 3 iteration 5
I/O process 3 iteration 6
I/O process 3 iteration 7
I/O process 3 iteration 8
I/O process 3 iteration 9
I/O-bound process 3 done
$ |
```

Test 4 – fork() Test

Run: forktest

```
mariu@DESKTOP-0K07MJA: ~$ balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-scsi-pci.0.force-io=on
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh

Fork enqueued 2
Queue 0: 2
Queue 1:
Queue 2:
Queue 3:
$ Forktest

Fork enqueued 3
Queue 0:
Queue 1:
Queue 2:
Queue 3:
Fork test

Fork enqueued 4
Queue 0:
Queue 1:
Queue 2:
Queue 3:

Fork enqueued 5
Queue 0: 5
Queue 1:
Queue 2:
Queue 3:

Fork enqueued 6
Queue 0: 6
Queue 1:
Queue 2:
Queue 3:

Fork enqueued 7
Queue 0: 7
Queue 1:
Queue 2:
Queue 3:

Fork enqueued 8
Queue 0: 7 8
Queue 1:
Queue 2:
Queue 3:

Fork enqueued 9
Queue 0: 8 9
Queue 1:
Queue 2:
Queue 3:

Fork enqueued 10
Queue 0: 9 10
Queue 1:
Queue 2:
Queue 3:

Fork enqueued 11
Queue 0: 9 10 11
Queue 1:
Queue 2:
Queue 3:

Fork enqueued 12
Queue 0: 11 12
Queue 1:
Queue 2:
Queue 3:

Fork enqueued 13
Queue 0: 11 12 13
Queue 1:
```

```
mariu@DESKTOP-0K07MJA: × + v

fork enqueued 52
Queue 0: 48 49 50 51 52
Queue 1:
Queue 2:
Queue 3:

fork enqueued 53
Queue 0: 49 50 51 52 53
Queue 1:
Queue 2:
Queue 3:

fork enqueued 54
Queue 0: 49 50 51 52 53 54
Queue 1:
Queue 2:
Queue 3:

fork enqueued 55
Queue 0: 50 51 52 53 54 55
Queue 1:
Queue 2:
Queue 3:

fork enqueued 56
Queue 0: 51 52 53 54 55 56
Queue 1:
Queue 2:
Queue 3:

fork enqueued 57
Queue 0: 51 52 53 54 55 56 57
Queue 1:
Queue 2:
Queue 3:

fork enqueued 58
Queue 0: 52 53 54 55 56 57 58
Queue 1:
Queue 2:
Queue 3:

fork enqueued 59
Queue 0: 54 55 56 57 58 59
Queue 1:
Queue 2:
Queue 3:

fork enqueued 60
Queue 0: 60
Queue 1:
Queue 2:
Queue 3:

fork enqueued 61
Queue 0:
Queue 1:
Queue 2:
Queue 3:

fork enqueued 62
Queue 0: 62
Queue 1:
Queue 2:
Queue 3:

fork enqueued 63
Queue 0: 62 63
Queue 1:
Queue 2:
Queue 3:

fork enqueued 64
Queue 0: 63 64
Queue 1:
Queue 2:
Queue 3:
fork test OK
$ |
```

Additional test:

```
$ quick  
fork enqueued 65  
Queue 0:  
Queue 1:  
Queue 2:  
Queue 3:  
  
process 65 ticks: 1  
ticks since boost: 2  
  
Quick process 65  
$
```

Github links:

<https://github.com/MariumAl/xv6-labs-2025>

https://github.com/FatimaMaskatia/Operating-System-Fall-2025/tree/MLFQ_Scheduler